Propagazione di fake news: simulazione della diffusione da un utente alla rete sociale

Patric Reineri, Daniela Magrì

6 giugno 2024

Indice

1 Introduzione e Metodologie adottate **3**

1.1 Comprendere il fenomeno della diffusione di fake news 3

1.2 Approccio utilizzato 4

1.2.1 Calcolo della probabilità di transizione da suscettibile a infetto 4

1.2.2 Calcolo della probabilità di transizione da infetto a recuperato 5

1.3 Utilizzo della libreria NDlib 5

1.4 Nota importante sulla simulazione 6

2 Analisi **7**

2.1 Cosa abbiamo analizzato 7

2.2 Confronto tra rete reale e rete artificiale 8

2.3 Sviluppo dei test e risultati ottenuti 11

2.4 Conclusioni 16

3 Visualizzazione **17**

3.1 Statica 17

3.2 Dinamica 20

CAPITOLO 1

Introduzione e Metodologie adottate

Il progetto si focalizza sulla simulazione della diffusione di fake news all'interno di una rete sociale, con particolare attenzione alla piattaforma Twitter. Partendo da un singolo individuo, identificato come un utente che può diffondere o confutare una fake news tramite un tweet o un retweet, analizziamo come l'informazione si propaga attraverso la rete sociale. L'obiettivo è comprendere le dinamiche e i meccanismi che favoriscono la diffusione delle fake news, nonché individuare le strategie per contrastarle efficacemente attraverso la scelta di un determinato insieme di nodi per avviare il processo di sfatamento.
Nella nostra simulazione il fenomeno della diffusione di fake news e quello di sfatamento coesistono e avvengono in parallelo ad ogni iterazione.
La nostra analisi studia i diversi modi per contrastare la diffusione di una fake news.

1.1 Comprendere il fenomeno della diffusione di fake news

Sui social media come Facebook o Twitter, ogni utente può potenzialmente raggiungere milioni di persone in pochi minuti, vengono utilizzati sempre più spesso per diffondere informazioni, fornendo un terreno fertile per bufale, contenuti di scarsa qualità e teorie del complotto. L'informazione, una volta divenuta virale, genera delle vere e proprie cascate di condivisioni, esponendo ripetutamente gli utenti agli stessi contenuti. L'esposizione continua ad un certo tipo di informazione può rafforzare alcuni pregiudizi[3], alimentando processi cognitivi come il ***confirmation bias***, ovvero la scelta mirata di alcune argomentazioni

per supportare e consolidare una propria opinione. Questo avviene quotidianamente sulle bacheche dei social network, che sono costruiti appositamente per connettere persone con le stesse idee o per ricostruire online comunità già esistenti, secondo il principio dell'***omofilia***, ovvero il fenomeno che vede aumentare la probabilità alla connessione reciproca tra persone che hanno interessi o caratteristiche simili[2].
Per la nostra simulazione abbiamo preso come riferimento la rete di twitter.

1.2 Approccio utilizzato

Per studiare il fenomeno di diffusione d'informazioni, l'approccio più comune è quello di ricorrere ai modelli che sono stati tradizionalmente usati nell'ambito della diffusione delle epidemie. In particolare, i modelli di base sono quelli compartimentali **SIR** e **SIS** in cui la popolazione è divisa in compartimenti che indicano lo stadio della malattia (*SIR = Susceptible, Infected, Recovered - SIS = Susceptible, Infected, Susceptible*) e l'evoluzione del processo è regolata da probabilità di transizione da uno stadio all'altro all'interno di un sistema di equazioni differenziali.

1.2.1 Calcolo della probabilità di transizione da suscettibile a infetto

Per regolare la transizione da suscettibili infetti abbiamo utilizzato una probabilità associata ad ogni nodo, questa è diversa per ogni nodo, secondo tre criteri:

1. P_i: si tratta di una probabilità di infezione associata al **grado di istruzione**. Infatti secondo la ricerca di Brent Berhart [1] riguardo al livello medio di istruzione degli utenti di *Twitter*, è emerso che il 13% degli utenti di *Twitter* ha il diploma di scuola superiore o inferiore, 24% la laurea universitaria, mentre il 32% è laureato con esperienza lavorativa (*il restante 21% non è specificato*).
   Abbiamo assegnato ad ogni nodo con una probabilità pesata in base a queste percentuali, il grado di istruzione associato. Successivamente a partire da questo abbiamo assegnato una probabilità di infezione diversa ad ogni nodo, seguendo il principio per cui un nodo meno istruito avrà probabilità maggiore di essere contagiato.

2. P_a: probabilità di infezione basata sugli anni di un nodo. Secondo gli studi di Brent Berhart [1], abbiamo derivato, secondo un semplice calcolo della media matematica,

che gli utenti *Twitter* hanno mediamente 32 anni. Pertanto **abbiamo assegnato un età ad ogni nodo secondo una distribuzione gaussiana con media 32 e normalizzata in modo tale da restituire un valore da 15 a 70 anni**.
A partire dall'età di ogni nodo è stata assegnata una probabilità P_a di infezione in base all'età secondo il principio per cui chi ha un'età tra 20 e 50 ha meno probabilità di essere contagiato rispetto a chi ha un età superiore a 50 anni o inferiore a 20.

3. P_c: credibilità della fake news. Nella nostra simulazione abbiamo assegnato **0.7** come probabilità di infettare sulla base del tipo di fake news.

La probabilità che il nodo i-esimo venga infettato è dato dalla media di queste tre probabilità.

1.2.2 Calcolo della probabilità di transizione da infetto a recuperato

Questa probabilità è stata assegnata tramite la definizione di un modello threshold eteregeneo, in cui ciascun nodo transita dallo stato di infetto a quello di *recovered* se la frazione dei suoi vicini *recovered* è superiore alla soglia associata al nodo.
Nella nostra simulazione abbiamo associato una soglia calcolata così:

$$\text{soglia_nodo} = \text{P_infezione_nodo} - 0.2$$

Il motivo è che un nodo che è stato infettato con probabilità alta, avrà una soglia più alta, poichè sarà più difficile da convincere che la notizia era falsa. *Abbiamo sottratto quel 0.2 come termine di normalizzazione*.

1.3 Utilizzo della libreria NDlib

NDlib è un pacchetto software Python che consente di descrivere, simulare e studiare i processi di diffusione su reti complesse.
In particolare per questo progetto è stato utilizzato il modello **SIR** sfruttando la libreria *NDlib*. Attraverso questa libreria è stato possibile creare un modello composto a compartimenti personalizzato. Abbiamo scelto due compartimenti per la nostra simulazione:

- **Node stochastic**: questo compartimento è servito per definire la transizione dal Compartimento ***suscettibili*** a quello degli **infetti** secondo una probabilità P.

Infatti durante la simulazione, preso un nodo che ha tra i vicini almeno un infetto, questo si infetterà secondo una probabilità P.

- **Node threshold**: questo compartimento è quello che regola la transizione da ***infetti*** a ***recuperati***. Il funzionamento è il seguente, preso un nodo che ha tra i vicini almeno un nodo nello stato *recovered*, questo transiterà da infetto a *recovered* qualora abbia una frazione di nodi vicini nello stato *recovered* tale da superare la soglia associata a quel nodo.

Attraverso la definizione di un modello composto è stato possibile aggiungere entrambi alla nostra simulazione. In questo modo il processo di diffusione e quello di sfatamento possono concorrere.

1.4 Nota importante sulla simulazione

Al fine di poter assegnare a ciascun nodo una probabilità di infezione diversa, basata sugli attributi, abbiamo definito una iterazione personalizzata del modello presente nel file `custom_iteration_bunch.py`.
Quello che facciamo è ad ogni iterazione, prendiamo lo stato attuale del modello e per ogni nodo suscettibile controlliamo se ha un vicino infetto e nel caso di esito positivo, calcoliamo una probabilità random di essere infetto, nel caso in cui questa superi la probabilità di essere infetto del nodo, questo transita da suscettibile ad infetto, infine aggiungiamo la modifica allo stato del modello. Successivamente procediamo facendo un iterazione del modello utilizzando la libreria *NDlib*.
Il procedimento appena descritto, ovvero far progredire sia gli infetti che i recovered ad ogni iterazione, consente di portare avanti all'interno di una iterazione anche il processo di sfatamento.

CAPITOLO 2

Analisi

2.1 Cosa abbiamo analizzato

Nella nostra analisi abbiamo simulato il processo di diffusione della fake news e di sfatamento della stessa su due reti:

- **Rete reale**: questa rete è stata creata sulla base del **dataset Higgs**. Il dataset Higgs è stato costruito dopo aver monitorato i processi di diffusione su Twitter prima, durante e dopo l'annuncio della scoperta di una nuova particella con le caratteristiche dell'elusivo bosone di Higgs il 4 luglio 2012. Sono stati presi in considerazione i messaggi postati su Twitter relativi a questa scoperta tra il 1° e il 7 luglio 2012.
  Nella rete generata i nodi sono gli utenti che hanno fatto un tweet in merito ad un argomento, invece un arco tra un nodo A e un nodo B viene creato se il nodo B retweetta un tweet del nodo A.
  Per motivi di complessità di calcolo e visualizzazione abbiamo fatto un *sampling* di 3000 nodi. Il *sampling* è stato fatto prendendo un nodo casuale nella rete ed espandendo a partire da esso i suoi vicini.

- **Rete artificiale**: questa rete è stata generata attraverso il modello di albert-barabasi utilizzando un numero di nodi uguale a quello della rete reale.

Al momento della simulazione è possibile scegliere la rete su cui eseguirla. Nella nostra analisi abbiamo analizzato prima la rete artificiale e successivamente quella reale.

2.2 Confronto tra rete reale e rete artificiale

Il confronto tra le due reti è stato svolto analizzando le proprietà strutturali di entrambe. Successivamente queste sono state comparate in diversi plot. Vediamo in dettaglio i vari confronti:

- **Confronto sul numero dei nodi, degli archi, connettività, average shortest path e diametro:**

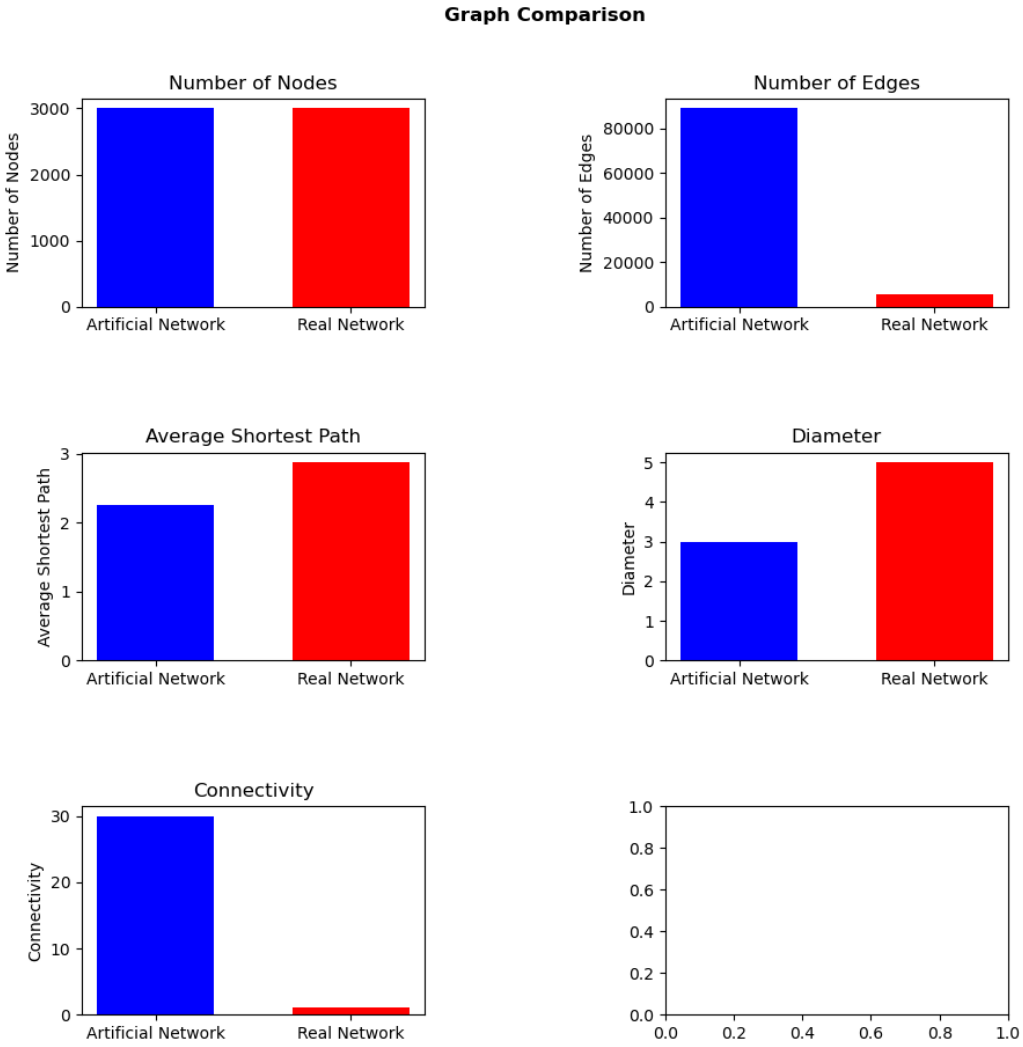


Figura 2.1

Le due reti risultano uguali nel numero dei nodi, simili nell'average shortest path, circa simili nel diametro. Tuttavia presentano differenze nella connettività e nel numero degli

archi.

- **Confronto sulla distribuzione del grado**:

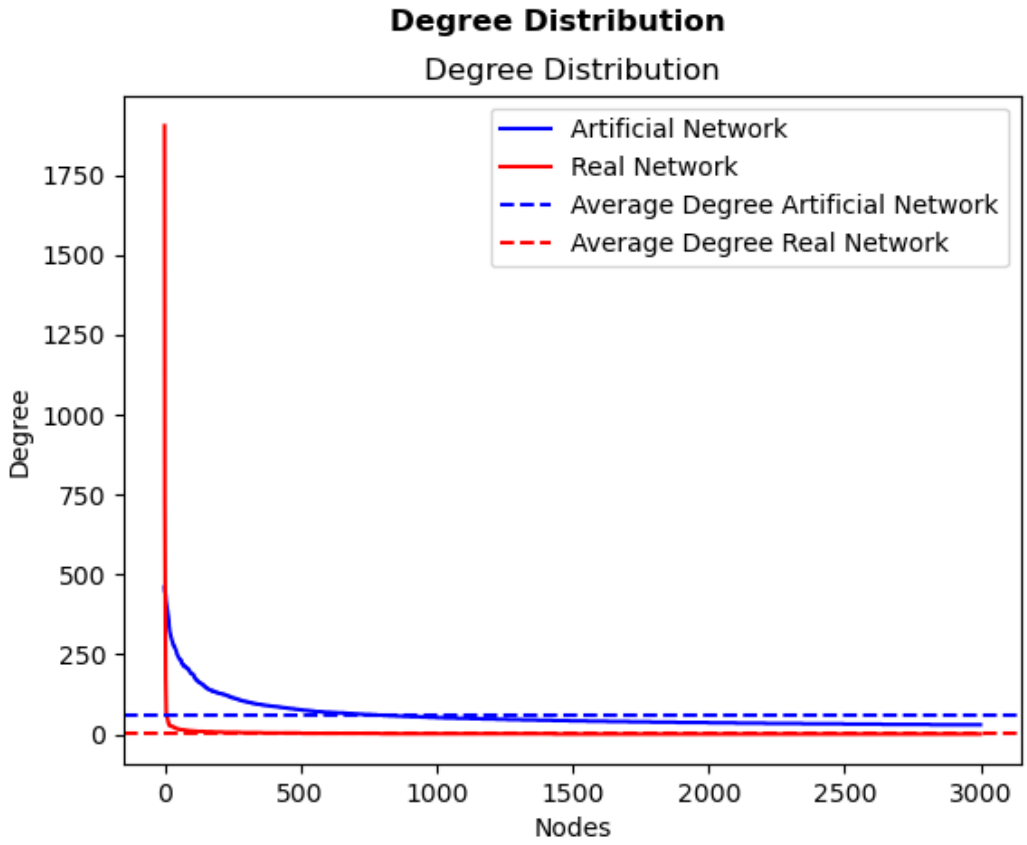


Figura 2.2

La rete artificiale riesce ad emulare la distribuzione a coda lunga, tuttavia non riesce a replicare i nodi che hanno degree maggiore di 500.

- **Confronto sui diversi tipi di centralità: degree centrality, betwennes centrality, closness centrality, betweenness centrality, eigenvector centrality**:

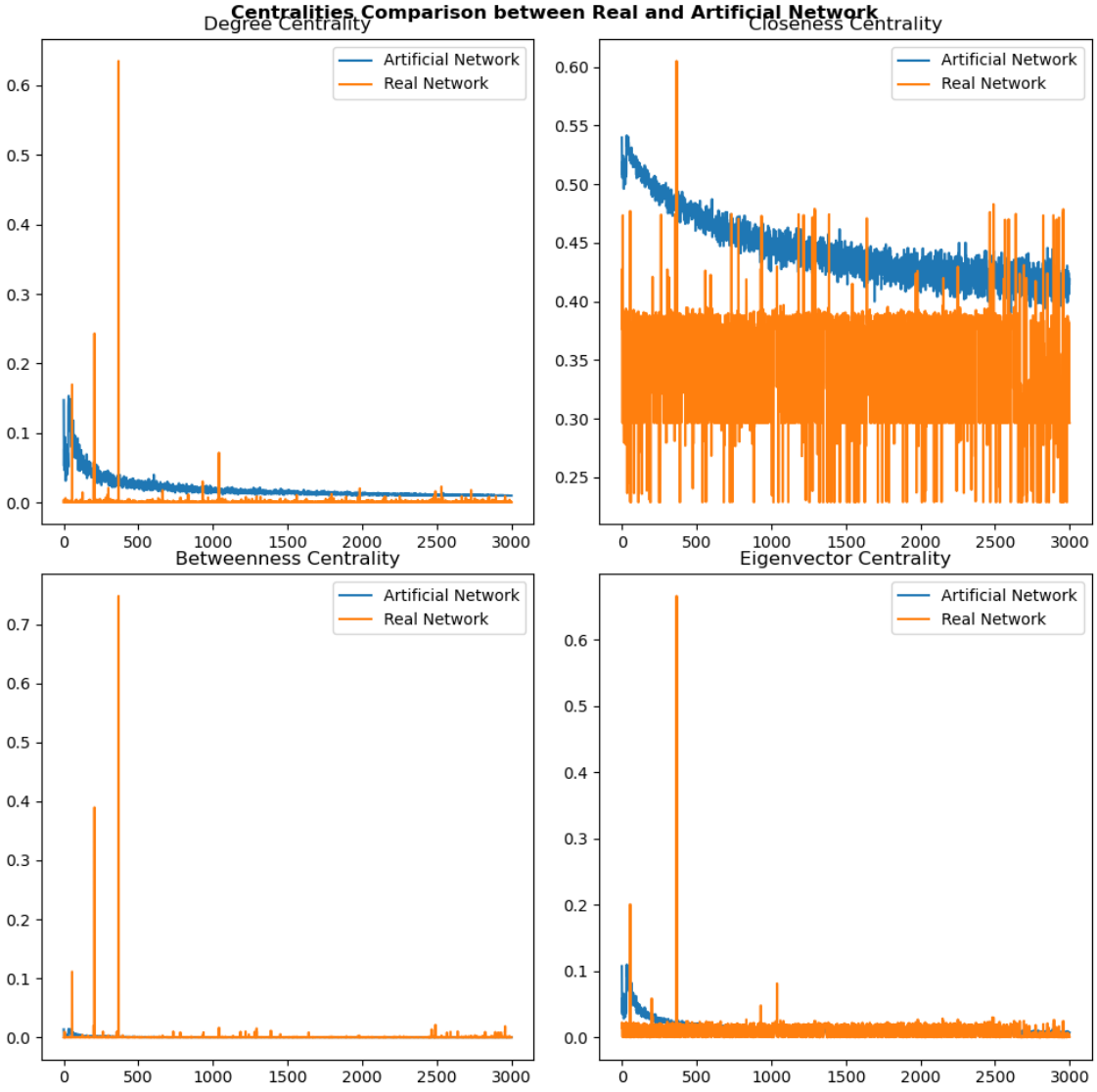


Figura 2.3

Possiamo vedere che la rete artificiale non riesce a replicare la centralità e i nodi altamente connessi presenti all'interno della rete reale, tuttavia approssima bene i nodi con *degree centrality* più basso.

Abbiamo invece approssimazioni peggiori nel caso del calcolo della *closeness*, non riuscendo ad emulare i nodi che consentono un alta diffusione delle informazioni, come nel caso della rete reale.

Analogamente per la *betweenness centrality*, la rete artificiale non riesce a catturare i nodi con alta *betweenness* presenti nella rete reale, non consentendo di replicare i nodi che fungono da ponte tra i vari gruppi di nodi.

Infine riesce ad approssimare i nodi con bassa *eigenvector centrality*, tuttavia non consen-

te di replicare quelli che hanno alta *eigenvector centrality*. Non possiamo avere quei nodi molto influente che sono invece presenti nella rete reale.

2.3 Sviluppo dei test e risultati ottenuti

Abbiamo deciso di sviluppare l'analisi delle reti in cinque test, per ciascuno di essi mostriamo una breve descrizione e i risultati ottenuti:

1. **Test 1:** el primo test abbiamo assegnato una frazione dello 0.08 % rispetto al totale dei nodi. Invece come inital adopter dello sfatamento abbiamo preso 120 hub, selezionati prendendo i 120 nodi con più alta degree centrality. Selezionando così i nodi più connessi nella rete. Come numero di iterazioni abbiamo scelto 50.
   Vediamo i risultati ottenuti:

   - Simulazione nella rete artificiale:

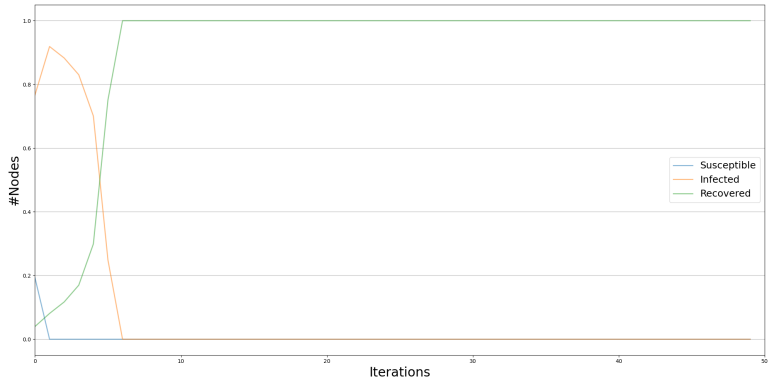


Figura 2.4

   - Simulazione nella rete reale:

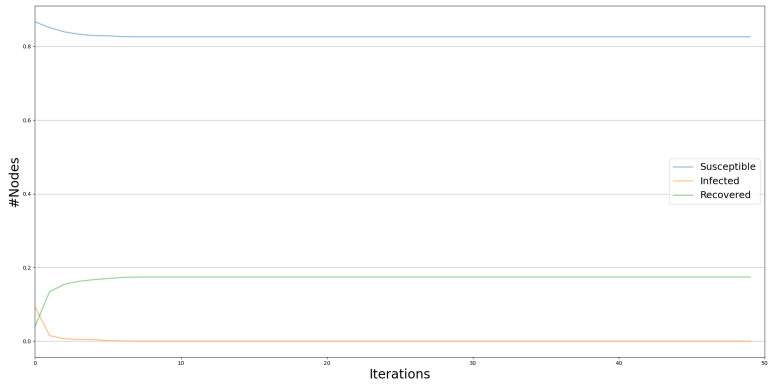


Figura 2.5

2. **Test 2:** nel secondo test abbiamo assegnato una frazione dello 0.08 % rispetto al totale dei nodi. Invece come ***inital adopter*** dello sfatamento abbiamo preso 120 nodi, selezionati prendendo i 120 nodi con più alta *betwennes centrality*, ovvero i nodi, selezionando così i nodi che fanno maggiormente da ponte tra gruppi di nodi per la diffusione di notizie. Come numero di iterazioni abbiamo scelto 50.
   Vediamo i risultati ottenuti:

   - Simulazione nella rete artificiale:

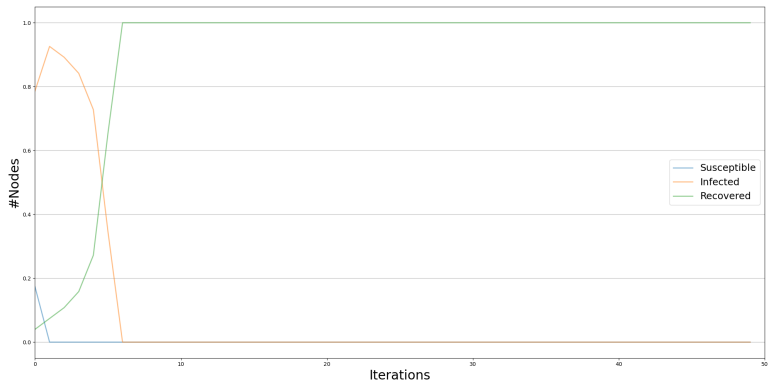


Figura 2.6

   - Simulazione nella rete reale:

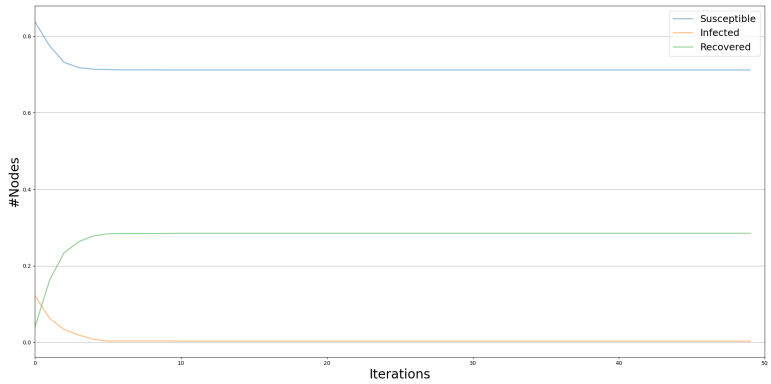


Figura 2.7

3. **Test 3:** Nel secondo test abbiamo assegnato una frazione dello 0.08 % rispetto al totale dei nodi. Invece come *inital adopter* dello sfatamento abbiamo preso 120 nodi, selezionati prendendo 120 nodi scelti in maniera randomica. Come numero di iterazioni abbiamo scelto 50.
   Vediamo i risultati ottenuti:

   - Simulazione nella rete artificiale:

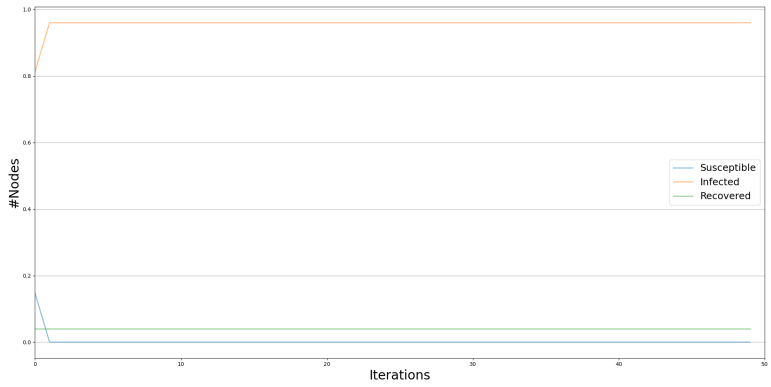


Figura 2.8

   - Simulazione nella rete reale:

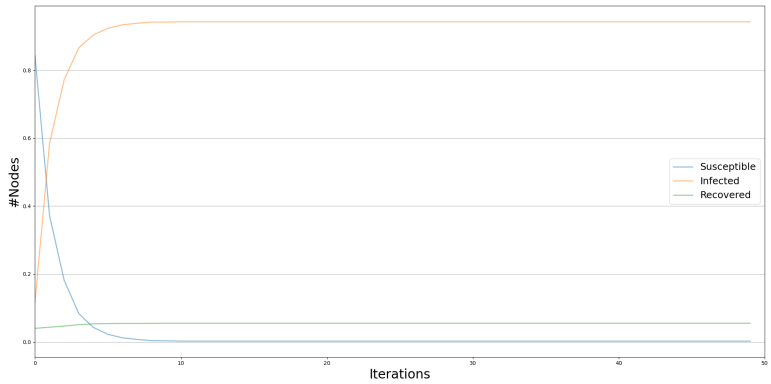


Figura 2.9

Possiamo notare che in entrambi i test, la percentuale di infetti prevale su quella dei *recovered*.

4. **Test 4:** nel secondo test abbiamo assegnato una frazione dello 0.08 % rispetto al totale dei nodi. Invece come *inital adopter* dello sfatamento abbiamo preso 120 nodi, selezionati prendendo i 120 nodi con più alta *closeness centrality*, ovvero i nodi, selezionando così i nodi che consentono di diffondere più rapidamente di altri la fake news o la notizia che la sfata. Come numero di iterazioni abbiamo scelto 50.
   Vediamo i risultati ottenuti:

   - Simulazione nella rete artificiale:

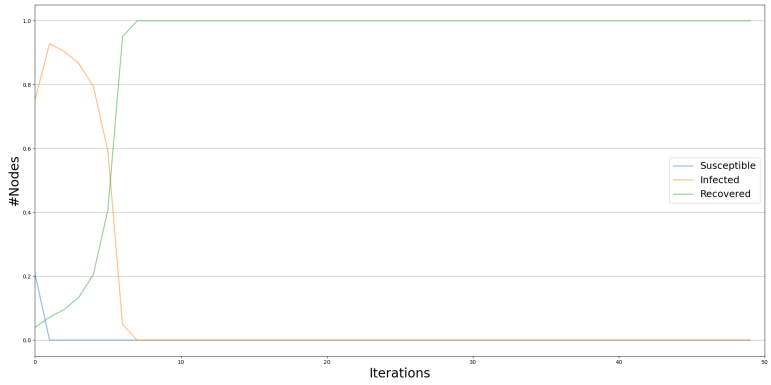


Figura 2.10

   - Simulazione nella rete reale:

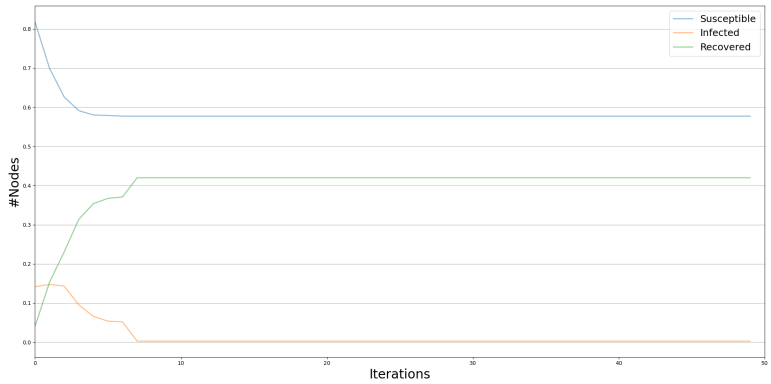


Figura 2.11

Test 5: nel secondo test abbiamo assegnato una frazione dello 0.08 % rispetto al totale dei nodi. Invece come *inital adopter* dello sfatamento abbiamo preso 120 nodi, selezionati prendendo i 120 nodi con più alta *eigenvector centrality*, selezionando così i nodi più influenti nella rete. Come numero di iterazioni abbiamo scelto 50.
Vediamo i risultati ottenuti:

- Simulazione nella rete artificiale:

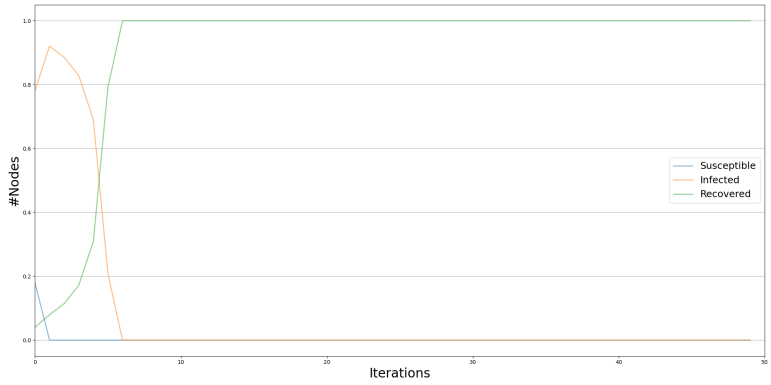


Figura 2.12

- Simulazione nella rete reale:

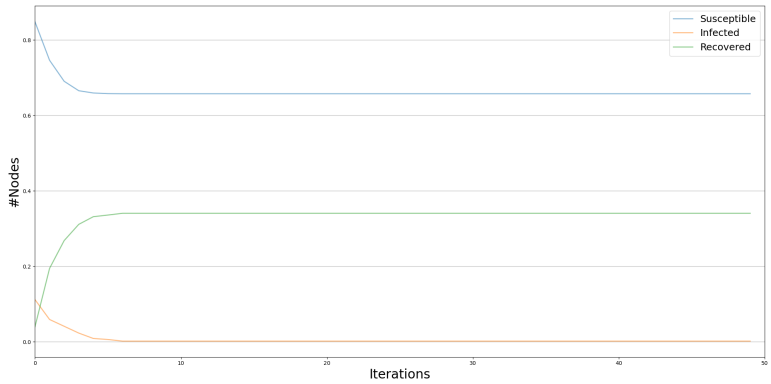


Figura 2.13

Nei test 1, 2, 4, 5 possiamo osservare che nel caso della rete artificiale, la maggior parte dei nodi suscettibili vengono infettati, successivamente il debunking dei nodi scelti come *initial adopter* ha avuto successo, sfatando la fake news. Nel caso di rete reale accade che i nodi sfatano la fake news più velocemente di quanto questa si diffonda, evitando il contagio.

2.4 Conclusioni

In conclusione il test che si è comportato meglio nella rete reale è stato il test 1, riuscendo a contrastare la diffusione della fake news più velocemente degli altri. Nella rete artificiale i test migliori per lo sfatamento sono stati il test 1, il test 2 e il test 5.

Nota sul numero di iterazioni, nella stampa ci fermiamo quando raggiungiamo uno stato stabile durante le iterazioni, ovvero dove nessun nodo ha possibilità di cambiare stato

CAPITOLO 3

Visualizzazione

La rappresentazione dei dati e della rete è realizzata attraverso diversi grafici, ciascuno dei quali enfatizza caratteristiche specifiche della nostra analisi.

3.1 Statica

La prima fase della visualizzazione si concentra sulla rappresentazione dei dati ottenuti dalle simulazioni. Questi dati consistono nel numero di individui (*nodi*), di una data rete, in uno stato (*'Infected'*, *'Recovered'*, *'Susceptible'*) specifico ad una particolare iterazione di un determinato test. Tutti questi sono parametri che l'utente che visualizza avrà la possibilità di specificare a run-time.
Vengono proposte diverse mappe a choropleth nella quale **il colore è utilizzato come *visual encoding*** per rappresentare la densità di individui nello stato visualizzato.
Una versione mostra solo le regioni, mentre un'altra include anche le città.
Inoltre, è possibile visualizzare queste mappe singolarmente o confrontare i diversi stati dei nodi in sequenza, ad esempio:

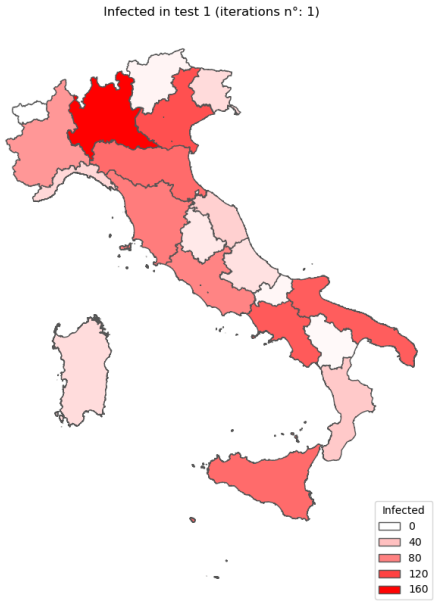


Figura 3.1: Esempio di choropleth in cui vengono mostrate solo le regioni.

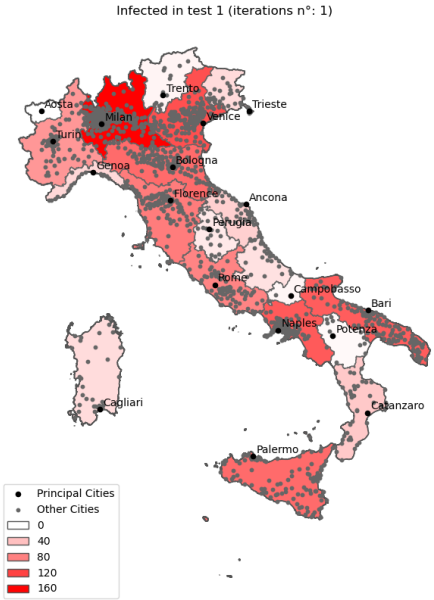
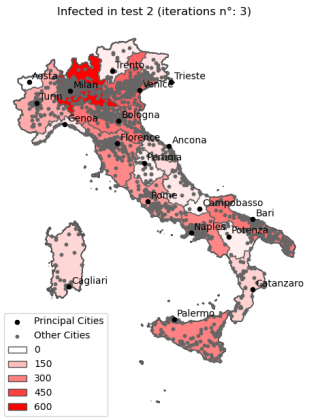
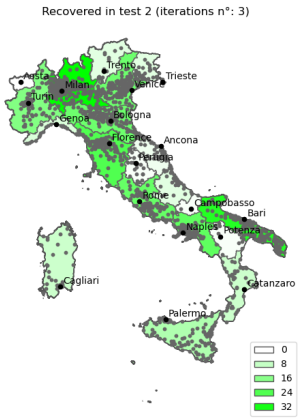
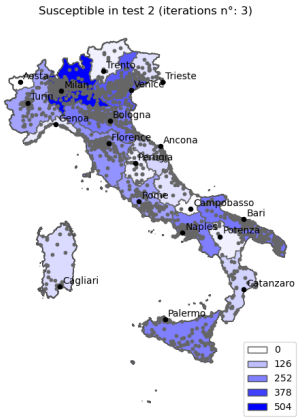
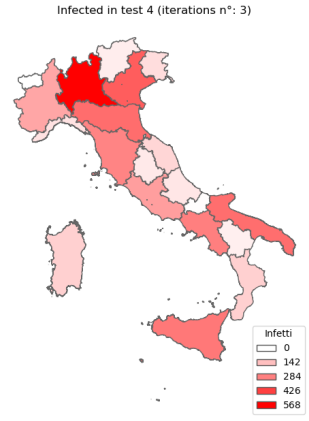


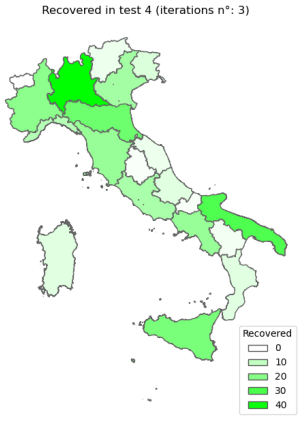
Figura 3.2: Esempio di choropleth in cui vengono mostrate anche le città.

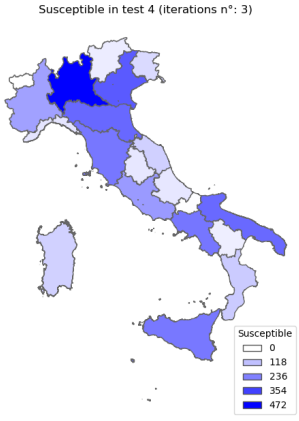

Infected in test 2 (iterations n°: 3)
Trento
Trieste
Aosta
Milan
Venice
Turin
Genoa
Bologna
Florence
Ancona
Perugia
Rome
Campobasso
Bari
Naples
Potenza
Cagliari
Catanzaro
Palermo
Principal Cities
Other Cities
0
150
300
450
600


Recovered in test 2 (iterations n°: 3)
Trento
Trieste
Aosta
Milan
Venice
Turin
Genoa
Bologna
Florence
Ancona
Perugia
Rome
Campobasso
Bari
Naples
Potenza
Cagliari
Catanzaro
Palermo
0
8
16
24
32


Susceptible in test 2 (iterations n°: 3)
Trento
Trieste
Aosta
Milan
Venice
Turin
Genoa
Bologna
Florence
Ancona
Perugia
Rome
Campobasso
Bari
Naples
Potenza
Cagliari
Catanzaro
Palermo
0
126
252
378
504


Infected in test 4 (iterations n°: 3)
Infetti
0
142
284
426
568


Recovered in test 4 (iterations n°: 3)
Recovered
0
10
20
30
40


Susceptible in test 4 (iterations n°: 3)
Susceptible
0
118
236
354
472

Queste rappresentazioni sono state create con `matplotlib` e `geopandas`.

3.2 Dinamica

Per la parte di visualizzazione dinamica abbiamo visualizzato la rete, utilizzando sempre come *visual encoding* il colore, permettendo all'utente di interagire in diversi modi con la visualizzazione.

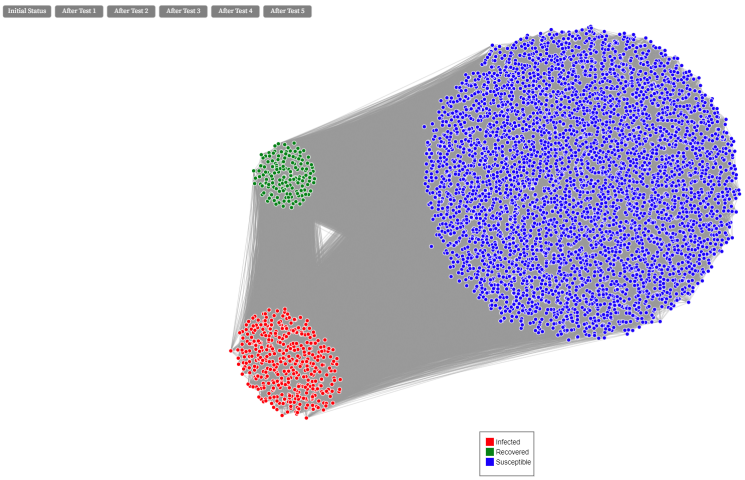


Figura 3.3: In questa particolare visualizzazione della rete vengono suddivisi i nodi in base al loro stato.

Ricordiamo che la visualizzazione è di due reti, in figura viene mostrata la rete artificiale.

Viene creato un centro attrattivo per ogni stato e il nodo viene attratto del centro del suo corrispettivo stato.

Come anticipato, la visualizzazione è interattiva: abbiamo implementato dei pulsanti che consentono all'utente di cambiare la visualizzazione in base ai risultati di un test da loro scelto. Inoltre è disponibile la funzionalità di *hovering* per mostrare gli attributi del nodo su cui è posizionato il cursore.

Non poteva infine mancare una visualizzazione geografica dinamica in cui visualizziamo i nodi mappati nello spazio:

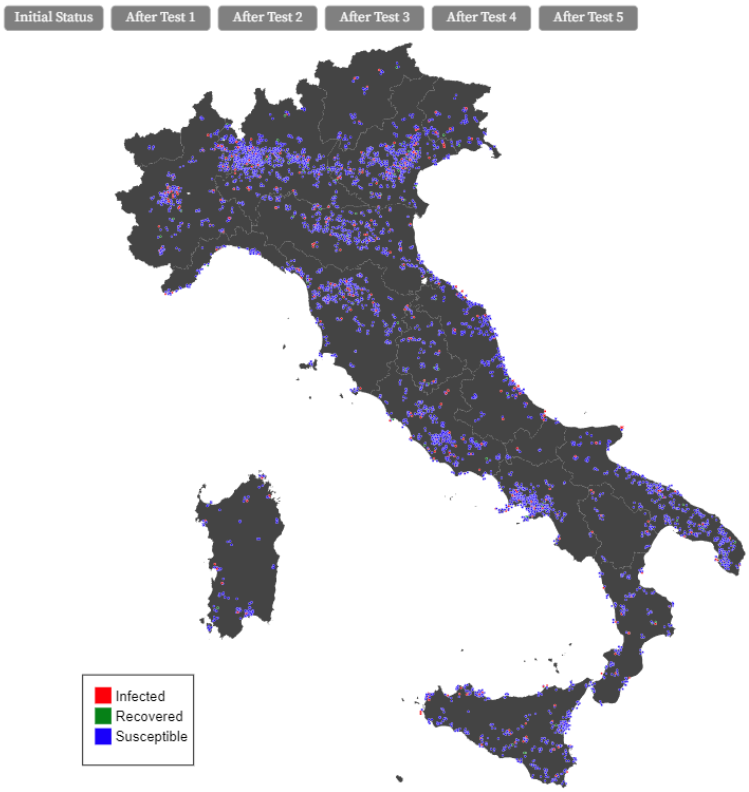
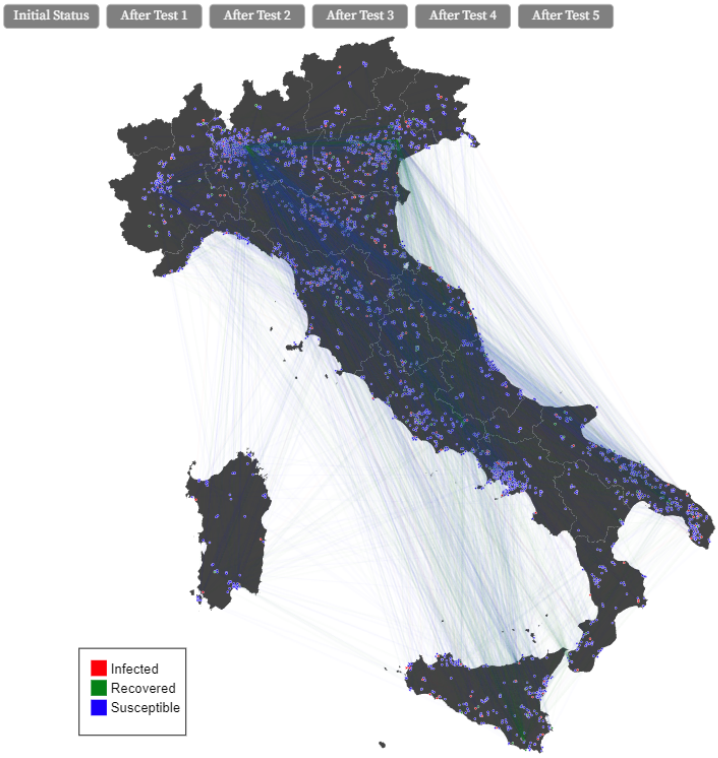


Figura 3.4

Ha anch'essa i pulsanti interattivi e la funzionalità di *hovering*. In aggiunta, la mappa di riferimento (*reference map*), è dotata di funzionalità di zoom, che permettono all'utente di ingrandire la visualizzazione sia utilizzando la rotellina del mouse sia cliccando su una determinata zona della mappa.
Quella mostrata in figura 3.4 è la versione senza archi, c'è anche la versione con gli archi:



Queste rappresentazioni sono state create con `d3.js`.

Nella cartella del progetto, ci sono ulteriori spiegazioni nel file `data_viz_first_part.ipynb`, che contiene dettagli, procedimenti, riferimenti e ragionamenti relativi alla prima parte della visualizzazione dei dati. Mentre all'interno della cartella `d3.js`, dove ci sono le rappresentazioni grafiche, esse sono accompagnate da spiegazioni e procedimenti.

Bibliografia

[1] Brent Barnhart. *Dati demografici dei social media per elaborare la strategia del tuo brand*.

[2] Marcella Tambuscio Giancarlo Ruffo. *Capire la diffusione della disinformazione e come contrastarla*.

[3] Robbie M. Sutton Aleksandra Cichocka Turchia Nefes Chee Siang Ang Farzin Deravi Karen M.Douglas, Joseph E. Uscinski. *Comprendere le teorie del complotto*.